My processing sound wave sketch line-by-line code explanation

Variables and arrays

```
String[] lines;
```

I create an array called `lines` that will store the lines of text from my CSV file. Each line is stored as a `String`, which means text.

```
SoundData[] sounds;
```

I create an array called `sounds` that will store my sound objects. Each object will contain information about one sound observation, including its loudness, location and type.

The setup() method

```
void setup()
```

I use `setup()` to run the instructions that prepare my sketch. Processing runs this method once when the sketch starts.

```
{
```

I use this opening brace to mark the beginning of the `setup()` method.

```
size(1600, 600);
```

I set the canvas size to 1600 pixels wide and 600 pixels high. This gives my sound wave plenty of horizontal space for all 100 observations.

```
lines = loadStrings("Sound data collection.csv");
```

I load my CSV file and store its contents in the `lines` array. Each row of the CSV is read as a line of text. The filename must match the file in my sketch's `data` folder.

```
sounds = new SoundData[lines.length - 1];
```

I create an array that will hold my sound objects. `lines.length` tells me how many lines are in the CSV. I subtract one because the first line contains the column headings, not a sound observation.

```
for (int i = 1; i < lines.length; i = i + 1)
```

I use a `for` loop to go through the CSV lines. I start at `i = 1` to skip the headings, continue while `i` is less than the number of lines, and increase `i` by one each time.

```
{
```

I mark the beginning of the loop's instructions.

```
String[] values = split(lines[i], ",");
```

I split the current CSV line into separate values wherever there is a comma. I store those values in an array called `values`, so I can access each column separately.

```
sounds[i - 1] = new SoundData(values);
```

I create a new `SoundData` object using the values from the current CSV row and store it in the `sounds` array. I use `i - 1` because my CSV loop starts at line 1, but my array starts at index 0.

```
}
```

I close the `for` loop.

```
}
```

I close the `setup()` method.

The draw() method

```
void draw()
```

I use `draw()` to control what appears on my canvas. Processing repeatedly runs this method while the sketch is running.

```
{
```

I mark the beginning of the `draw()` method.

```
background(255);
```

I make the background white. The value `255` represents white in Processing's greyscale colour system.

```
stroke(30);
```

I set the colour of the lines to a very dark grey. This makes the waveform stand out against the white background.

```
strokeWeight(9);
```

I set the thickness of the lines to 9 pixels. This gives my waveform the bold, blocky appearance I wanted.

```
for (int i = 0; i < sounds.length; i = i + 1)
```

I use another `for` loop to go through every sound object in my `sounds` array. I start at index 0 and increase `i` by one until every observation has been drawn.

```
{
```

I mark the beginning of the loop's instructions.

```
float x = map(i, 0, sounds.length, 30, width - 30);
```

I calculate the horizontal position of each sound observation. The `map()` method converts the observation's index into a position across the canvas, leaving a 30-pixel margin at each side. I store the result in a `float` variable called `x`.

```
float barHeight = map(sounds[i].loudness, 1, 10, 12, height / 2 - 30);
```

I calculate the height of the waveform bar using that sound's loudness. The `map()` method converts loudness values from the range 1–10 into bar heights. Quieter sounds produce shorter bars, while louder sounds produce taller bars.

```
line(x, height / 2 - barHeight,
```

I start drawing a line at horizontal position `x`. The first vertical coordinate is the centre of the canvas minus the bar height, which places the top of the bar above the centre.

```
x, height / 2 + barHeight);
```

I finish the line at the same horizontal position, but this time the vertical coordinate is the centre plus the bar height. This places the bottom of the bar below the centre. Together, these coordinates create one vertical bar that extends equally above and below the centre line.

```
}
```

I close the loop. The loop repeats the drawing instructions for every sound observation.

```
}
```

I close the `draw()` method.

The SoundData class

```
class SoundData
```

I create my own class called `SoundData`. It defines the information that I want to store for each sound observation.

```
{
```

I mark the beginning of the class.

```
float loudness;
```

I create a variable called `loudness` to store the loudness rating for a sound. I use `float` because it stores numerical values.

```
String location;
```

I create a variable called `location` to store where the sound was heard. I use `String` because the location is text.

```
String type;
```

I create a variable called `type` to store the category of the sound, such as a vehicle or nature sound. I use `String` because the category is text.

The constructor

```
SoundData(String[] values)
```

I create a constructor for the `SoundData` class. It receives the array of values from one CSV row and uses them to fill the object's variables.

```
{
```

I mark the beginning of the constructor.

```
location = values[2];
```

I store the location from the third CSV column in the object's `location` variable. Array positions start at 0, so `values[2]` refers to the third column.

```
loudness = float(values[3]);
```

I take the loudness value from the fourth CSV column and convert it from text into a number using `float()`. This allows me to use it in calculations to determine the height of the waveform.

```
type = trim(values[4]);
```

I store the sound type from the fifth CSV column. The `trim()` method removes any unnecessary spaces at the beginning or end of the text.

```
}
```

I close the constructor.

```
}
```

I close the `SoundData` class.

How the whole sketch works

I load my CSV file, turn each observation into a `SoundData` object, and store the objects in an array. Processing then goes through the objects and draws one vertical bar for each observation. The position of each bar depends on its place in the dataset, while its height depends on the sound's loudness.

My sketch therefore turns my collection of 100 everyday sound observations into a visual waveform. The bars are a representation of my recorded loudness values, rather than recordings of the actual sounds.

Important terms to remember

- **Array:** Stores multiple values or objects in one variable.
- **Class:** Defines the information and structure for objects.
- **Object:** An individual instance of a class, representing one sound observation.
- **Constructor:** Sets up a new object using the information supplied to it.
- **Loop:** Repeats instructions for multiple observations.

- **`loadStrings()`:** Loads lines of text from a file.
- **`split()`:** Separates a line of text into individual values.
- **`float()`:** Converts text representing a number into a numerical value.
- **`map()`:** Converts a value from one numerical range to another.
- **`line()`:** Draws a line between two coordinates.
- **`trim()`:** Removes unnecessary spaces from the ends of text.